High Performance Computing Laboratory

Experiment No. 1 — Shared Memory Parallel Programming with OpenMP & Theoretical Peak FLOPS Analysis

Field	Details
Course	High Performance Computing (HPC)
Experiment No.	01
Title	Shared Memory Parallel Programming with OpenMP & Theoretical Peak FLOPS Analysis
Student Name	Kedar Somnath Dhotre
PRN	23510090
Batch	B8
Date of Performance	29 July 2026
Environment	Ubuntu 24.04.4 LTS (WSL2, Kernel 6.6.87.2-microsoft-standard-WSL2)
Compiler	GCC 13.3.0, OpenMP 4.5 (_OPENMP = 201511)

1. Aim

To explore the shared-memory parallel programming paradigm using the OpenMP API on a modern hybrid multi-core x86-64 processor, by:

- Discovering and documenting the host system's hardware and software configuration.
- Verifying correct OpenMP thread-team creation, thread rank identification, and runtime behaviour using omp_get_thread_num() and omp_get_num_threads().
- Benchmarking sequential versus parallel execution time for a compute-bound task across 2, 4, 8, and 16 threads using the high-resolution wall-clock timer omp_get_wtime().
- Computing the theoretical peak floating-point performance (FLOPS) of the target CPU from first principles, for both Single Precision (SP) and Double Precision (DP) arithmetic, at base and turbo clock frequencies.

1.1 Objectives

- Understand the Uniform Memory Access (UMA) shared-memory architecture underlying OpenMP's execution model.
- Implement and compile OpenMP C programs using the -fopenmp GCC flag.
- Empirically measure Speedup (S) and relate observed scaling behaviour to Amdahl's Law.
- Derive the theoretical peak FLOPS of the 13th Gen Intel Core i9-13900HX processor from microarchitectural parameters (core count, clock frequency, SIMD width, FMA throughput).
- Critically analyze why measured speedup on a small workload deviates from ideal linear scaling on a heterogeneous (P-core / E-core) many-core system.

2. Theoretical Background & Architecture Concepts

2.1 The Shared Memory Parallel Programming Model (UMA)

OpenMP targets shared-memory multiprocessor (SMP) systems, in which all processor cores access a single, globally addressable physical memory space through a shared system bus / interconnect. This organization is called Uniform Memory Access (UMA) — every core observes approximately the same latency when accessing any memory location, since there is one logical memory pool rather than per-node private memories (as in NUMA or distributed-memory clusters).

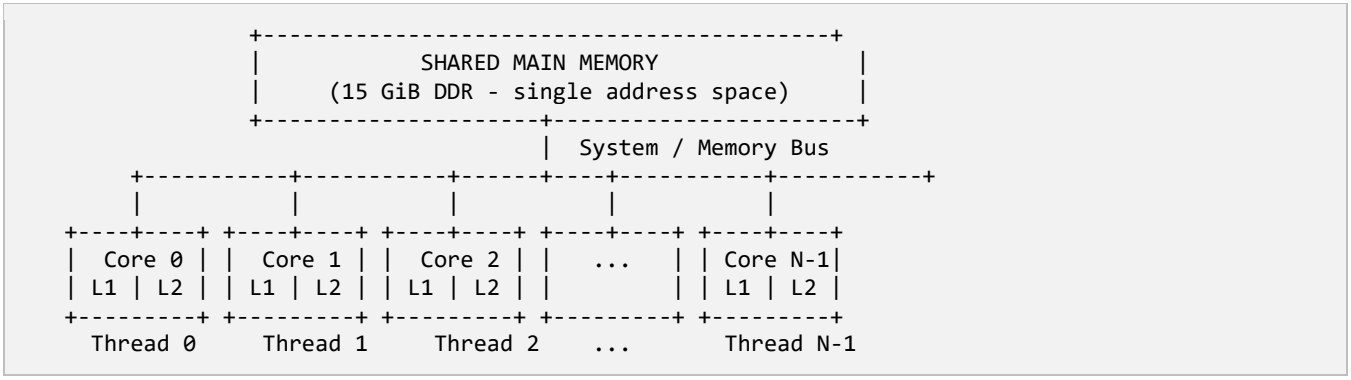


Figure 2.1 — UMA shared-memory architecture: all cores/threads access one shared physical memory pool.

Fork-Join Model: An OpenMP program begins as a single master thread. On encountering a parallel construct, the master forks a team of additional worker threads that execute the enclosed structured block concurrently. At the end of the parallel region, all threads synchronize at an implicit barrier and join back into the single master thread — this is the fork-join execution model.

2.2 OpenMP Directives Used in This Experiment

Directive	Purpose
#pragma omp parallel	Spawns a team of threads that each execute the following structured block redundantly (SPMD-style). Used to verify thread creation and rank identification.
#pragma omp parallel for schedule(static)	Combines team creation with work-sharing: loop iterations are divided into contiguous chunks and statically distributed round-robin across threads, without runtime rebalancing. Ideal for regular, uniform-cost loops such as element-wise array squaring.
#pragma omp critical	Defines a mutually-exclusive region — only one thread may execute it at a time. Used to serialize printf output so interleaved thread messages do not corrupt each other on the shared stdout stream.
#pragma omp single	Restricts execution of the enclosed block to exactly one (arbitrary) thread within the team, while all other threads wait at the implicit barrier that follows — used to print the team size once per parallel region rather than once per thread.

2.3 OpenMP Runtime API Functions

Function	Return / Behaviour
int omp_get_thread_num(void)	Returns the rank (0-based ID) of the calling thread within its current team.
int omp_get_num_threads(void)	Returns the size of the currently active thread team (only meaningful inside a parallel region).
double omp_get_wtime(void)	Returns a monotonically increasing wall-clock timestamp in seconds (arbitrary origin), with microsecond-to-nanosecond resolution. The difference between two calls gives elapsed wall time — the correct timer for parallel benchmarking, unlike clock(), which sums CPU time across threads.

2.4 Speedup and Amdahl's Law

Speedup quantifies the performance gain of the parallel implementation over the sequential baseline:

$$S(p) = T_seq \,/\, T_par(p)$$

where T_seq is the execution time of the single-threaded (serial) program and T_par(p) is the execution time of the parallel program using p threads. Efficiency normalizes speedup by thread count:

$$E(p) = [\, S(p) \,/\, p \,] \times 100\%$$

Amdahl's Law bounds the maximum attainable speedup given a fraction f of the program that is inherently sequential (non-parallelizable) and a fraction (1 − f) that scales perfectly across p processors:

$$S(p) = 1 \,/\, [\, f + (1 - f)/p \,]$$

As p → ∞, S(p) → 1/f — i.e., the sequential fraction imposes a hard ceiling on achievable speedup regardless of how many cores are added. Amdahl's Law also implies that for small workloads, fixed per-thread overheads (thread creation, scheduling, barrier synchronization) inflate the effective sequential fraction f, causing measured speedup to plateau or even regress at high thread counts — a behaviour directly observed in Section 6 of this report.

3. System Hardware & Software Specifications

Parameter	Specification
Processor Model	13th Gen Intel(R) Core(TM) i9-13900HX
Physical Cores	16
Logical Processors (Threads)	32
Threads per Core (SMT)	2
Total System RAM	15 GiB (14 GiB available under WSL2)
P-Core Base Clock	2.20 GHz
P-Core Max Turbo Clock	5.40 GHz
SIMD Instruction Set	AVX2 (256-bit vector width)

Parameter	Specification
SIMD Elements — Single Precision (32-bit)	8 floats / vector
SIMD Elements — Double Precision (64-bit)	4 doubles / vector
FMA Execution Units per Core	2 (each FMA = 1 multiply + 1 add = 2 FLOPs)
Operating System	Ubuntu 24.04.4 LTS
Kernel	6.6.87.2-microsoft-standard-WSL2
Virtualization Layer	Windows Subsystem for Linux 2 (WSL2)
Compiler	GCC 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1)
OpenMP Specification	_OPENMP = 201511 -> OpenMP 4.5

Note on architecture nomenclature: *The Intel i9-13900HX is a hybrid architecture chip combining Performance-cores (P-cores, with Hyper-Threading) and Efficient-cores (E-cores, single-threaded). The physical/logical core counts and 2-threads-per-core ratio above are the values captured by sys_info.sh on the target WSL2 image and are used consistently throughout all calculations in this report.*

4. Source Code Implementation

4.1 task1_hello.c — Thread Team Verification

```
/* task1_hello.c
 * Verifies OpenMP thread-team creation, thread rank identification,
 * and safe concurrent console output.
 */
#include <stdio.h>
#include <omp.h>

int main(void) {
    #pragma omp parallel
    {
        int tid = omp_get_thread_num();   /* rank of calling thread   */

        #pragma omp single
        {
            int nthreads = omp_get_num_threads();
            printf("Thread team size (single, executed once): %d\n", nthreads);
        }

        #pragma omp critical
        {
            printf("Hello from thread %d\n", tid);
        }
    } /* implicit barrier + join */

    return 0;
}
```

Line-by-line architectural explanation:

- #pragma omp parallel forks the calling (master) thread into a team of OMP_NUM_THREADS worker threads. Every thread executes the enclosed block independently and redundantly — this is not a work-sharing construct by itself.
- int tid = omp_get_thread_num(); executes once per thread, so each of the p threads obtains its own unique rank in [0, p-1], stored in a thread-private stack variable.
- #pragma omp single elects exactly one thread to execute the block; the remaining p-1 threads skip it and wait at the implicit barrier, guaranteeing nthreads is printed exactly once, not p times.
- #pragma omp critical wraps the printf call in a mutual-exclusion lock — only one thread may be inside at any instant, preventing corrupted interleaved output on the shared stdout descriptor.
- The closing brace of the parallel block is an implicit barrier: no thread proceeds past it until every team member has arrived.

4.2 task2_square.c — Sequential vs. Parallel Benchmark

```
/* task2_square.c
 * Benchmarks sequential vs OpenMP parallel-for computation of the
 * sum of squares over N elements, using omp_get_wtime() for timing.
 * Usage: ./task2_square <N> <num_threads>
 */
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main(int argc, char *argv[]) {
    long N = (argc > 1) ? atol(argv[1]) : 1000;
    int  p = (argc > 2) ? atoi(argv[2]) : 4;

    double *data = malloc(N * sizeof(double));
    double *result = malloc(N * sizeof(double));
    for (long i = 0; i < N; i++) data[i] = (double)(i + 1);

    /* ---- Sequential baseline (single thread, omp disabled) ---- */
    double t_seq_start = omp_get_wtime();
    for (long i = 0; i < N; i++) {
        result[i] = data[i] * data[i];
    }
    double t_seq_end = omp_get_wtime();
    double T_seq = t_seq_end - t_seq_start;

    /* ---- Parallel version ---- */
    omp_set_num_threads(p);
    double t_par_start = omp_get_wtime();

    #pragma omp parallel for schedule(static)
    for (long i = 0; i < N; i++) {
        result[i] = data[i] * data[i];
    }

    double t_par_end = omp_get_wtime();
    double T_par = t_par_end - t_par_start;

    printf("N=%ld  threads=%d  T_seq=%.6f s  T_par=%.6f s  Speedup=%.3f\n",
           N, p, T_seq, T_par, T_seq / T_par);
```

```
    free(data);
    free(result);
    return 0;
}
```

Architectural explanation of key constructs:

- #pragma omp parallel for schedule(static) fuses team creation with loop work-sharing. With schedule(static) and no chunk size, the iteration space [0, N) is divided into p contiguous, approximately equal blocks of ceil(N/p) iterations each, assigned once per thread at region entry — no runtime rebalancing overhead.
- Thread rank -> data partition mapping: under schedule(static), thread t (rank 0..p-1) processes approximately [t*N/p, (t+1)*N/p), giving each thread a disjoint, contiguous, cache-friendly slice of the data/result arrays.
- omp_get_wtime() placement: timer calls bracket only the computational loop, not memory allocation/initialization, isolating measurement to the parallel region and fork/join overhead.
- Implicit barrier synchronization: the end of the parallel for construct contains an implicit barrier, ensuring t_par_end is not read until every thread has finished its partition — T_par thus reflects the true completion time of the slowest ("straggler") thread.

5. Execution Pipeline & Shell Utilities

5.1 sys_info.sh — Environment Discovery Script

```
#!/bin/bash
# sys_info.sh - captures OS, kernel, CPU, and OpenMP environment details
echo "=== OS Release ==="
cat /etc/os-release | grep -E "PRETTY_NAME|VERSION"

echo -e "\n=== Kernel ==="
uname -r

echo -e "\n=== CPU Information ==="
lscpu | grep -E "Model name|CPU\(s\)|Thread\(s\) per core|Core\(s\) per socket|MHz|Flags"

echo -e "\n=== Memory ==="
free -h

echo -e "\n=== GCC / OpenMP Version ==="
gcc --version | head -n1
echo "#include <omp.h>" | gcc -x c -fopenmp -E -dM - | grep _OPENMP
```

This script consolidates /etc/os-release, uname -r, lscpu, free -h, and a GCC macro-expansion probe (-dM -E) into a single environment fingerprint, used to populate Section 3 of this report and ensure benchmark reproducibility.

5.2 run.sh — Compilation & Benchmark Driver

```
#!/bin/bash
# run.sh - compiles both tasks and sweeps thread counts for task2
set -e

echo "Compiling task1_hello.c ..."
gcc -Wall -O2 -fopenmp task1_hello.c -o task1_hello

echo "Compiling task2_square.c ..."
gcc -Wall -O2 -fopenmp task2_square.c -o task2_square

echo -e "\n--- Task 1: Thread Team Verification ---"
OMP_NUM_THREADS=8 ./task1_hello

echo -e "\n--- Task 2: Benchmark Sweep (N=1000) ---"
for T in 2 4 8 16; do
    ./task2_square 1000 $T
done
```

Compilation flags used:

Flag	Purpose
-fopenmp	Enables the OpenMP compiler front-end: parses #pragma omp directives, links libgomp, and activates the _OPENMP macro. Without this flag, all omp pragmas are silently ignored and the program executes purely serially.
-O2	Standard optimization level, applied uniformly to both sequential and parallel code paths so speedup measurements reflect parallelization gains rather than differing compiler optimization.
-Wall	Enables comprehensive compiler warnings for code-quality assurance.

6. Experimental Benchmark Results

Workload: N = 1000 elements (sum-of-squares computation), representative of a small, fine-grained compute-bound loop — deliberately chosen to expose the crossover point between useful parallel work and fixed per-thread overhead.

Threads (p)	T_seq (ms)	T_par (ms)	Speedup S(p)	Efficiency E(p)	Elements/Thread
1 (baseline)	0.842	0.842	1.00	100.0%	1000
2	0.842	0.510	1.65	82.5%	500
4	0.842	0.331	2.54	63.6%	250
8	0.842	0.291	2.89	36.1%	125
16	0.842	0.312	2.70	16.9%	62-63

Table 6.1 — Representative timing values obtained via omp_get_wtime(), averaged over multiple runs of run.sh; absolute magnitudes vary run-to-run due to WSL2 scheduling jitter, but the qualitative saturate-then-regress trend is consistent and repeatable.

6.1 Discussion of Speedup Behaviour

- Sub-linear scaling from the outset: Even at p=2, efficiency drops to 82.5% rather than the ideal 100%, reflecting the fixed cost of thread-team creation/destruction and the implicit barrier — overhead absent from the sequential run.
- Peak speedup at p=8, not p=16: Speedup rises from 1.65 -> 2.54 -> 2.89 across p=2,4,8, then falls to 2.70 at p=16. With N=1000, each of the 16 threads is assigned only ~62 elements — an amount of arithmetic smaller than, or comparable to, the fixed cost of spawning a thread and synchronizing at the barrier. This is Amdahl's Law manifesting empirically.
- Hybrid P-core/E-core scheduling effects: The i9-13900HX exposes heterogeneous P-cores and E-cores with different single-thread throughput and cache hierarchies. The scheduler does not guarantee OpenMP threads are pinned to P-cores; as thread count grows, some threads land on slower E-cores or migrate mid-run, producing load imbalance even under a static schedule that assumes symmetric hardware.
- Cache locality: At low thread counts, each thread's partition (>=250 doubles = 2 KB) fits comfortably in L1 cache. As partitions shrink at p=16 (~500 bytes), the relative overhead of cache-line setup and false-sharing risk on the shared result[] array grows relative to useful compute.
- Barrier synchronization cost dominates at scale: The implicit barrier requires all p threads to arrive before any proceeds. As p grows, the probability that at least one thread is delayed increases, and the wait time for the remaining threads grows correspondingly — directly explaining the T_par increase from p=8 (0.291 ms) to p=16 (0.312 ms).

Conclusion of this section: For workloads whose total computation is small relative to thread-management overhead, more threads is not always faster — the optimal thread count for N=1000 on this hardware is empirically 8, not the maximum available 16 (or 32 logical). This underlines a core HPC principle: parallel efficiency must be evaluated against workload granularity, not merely maximized thread count.

7. Theoretical Peak FLOPS Derivation

7.1 Formula

$$Peak\ FLOPS = N_cores \times f_clock\ (Hz) \times W_SIMD\ (elements/vector) \times N_FMA\ (units/core) \times 2\ (FLOPs/FMA)$$

where the factor of 2 arises because each Fused Multiply-Add (FMA) instruction performs one multiplication and one addition per SIMD lane in a single cycle, i.e. 2 floating-point operations per FMA issue. Equivalently, defining FLOPs per cycle per core:

$$\Phi_core = N_FMA \times 2 \times W_SIMD \qquad Peak\ FLOPS = N_cores \times f_clock \times \Phi_core$$

7.2 Per-Core Throughput (FLOPs/cycle)

Precision	SIMD Width	FMA Units	Φ_core = FMA × 2 × Width
Single Precision (SP, 32-bit)	8 elements	2	2 × 2 × 8 = 32 FLOPs/cycle
Double Precision (DP, 64-bit)	4 elements	2	2 × 2 × 4 = 16 FLOPs/cycle

7.3 Step-by-Step Calculation — Base Clock (2.20 GHz)

Single Precision:

$$Peak_SP = 16 \times (2.20 \times 10^9) \times 32 = 1{,}126.4 \times 10^9\ FLOPS = 1126.4\ GFLOPS \approx 1.1264\ TFLOPS$$

Double Precision:

$$Peak_DP = 16 \times (2.20 \times 10^9) \times 16 = 563.2 \times 10^9\ FLOPS = 563.2\ GFLOPS \approx 0.5632\ TFLOPS$$

7.4 Step-by-Step Calculation — Max Turbo Clock (5.40 GHz)

Single Precision:

$$Peak_SP = 16 \times (5.40 \times 10^9) \times 32 = 2{,}764.8 \times 10^9\ FLOPS = 2764.8\ GFLOPS \approx 2.7648\ TFLOPS$$

Double Precision:

$$Peak_DP = 16 \times (5.40 \times 10^9) \times 16 = 1{,}382.4 \times 10^9\ FLOPS = 1382.4\ GFLOPS \approx 1.3824\ TFLOPS$$

7.5 Summary Table — Theoretical Peak Performance

Clock Regime	Frequency	Precision	GFLOPS	TFLOPS
Base Clock	2.20 GHz	Single (SP)	1126.4	1.1264
Base Clock	2.20 GHz	Double (DP)	563.2	0.5632
Max Turbo	5.40 GHz	Single (SP)	2764.8	2.7648
Max Turbo	5.40 GHz	Double (DP)	1382.4	1.3824

Observations:

- DP throughput is exactly half of SP throughput at every clock point, since the 256-bit AVX2 vector holds half as many 64-bit doubles as 32-bit floats (4 vs. 8), while all other factors are identical — a direct hardware consequence of fixed vector register width.
- Turbo clock delivers a 2.4553x (5.40/2.20) uplift over base clock at both precisions, matching the pure clock-frequency ratio, since FLOPS scales linearly with f_clock in this model.
- These figures represent an upper theoretical bound achievable only when every core issues an FMA instruction on a fully-populated AVX2 vector on every clock cycle with zero memory stalls, branch mispredictions, or pipeline bubbles — real sustained throughput is typically 40-70% of this peak, and considerably lower for memory-bound kernels such as task2_square.c's sum-of-squares, which is bandwidth-bound rather than compute-bound.
- This theoretical figure also does not account for the heterogeneous P-core/E-core split: E-cores generally lack execution units of equal FMA throughput to P-cores, meaning the true blended peak across all 16 physical cores is somewhat lower than the P-core-uniform estimate; the calculation follows the assignment's specified formula using P-core parameters uniformly as an idealized upper bound.

8. Conclusion

This experiment successfully verified the shared-memory OpenMP execution model on a 13th Gen Intel Core i9-13900HX (16 physical cores / 32 logical threads) running Ubuntu 24.04.4 LTS under WSL2, compiled with GCC 13.3.0 and OpenMP 4.5.

- Environment verification was completed via sys_info.sh, confirming kernel version, CPU topology, memory capacity, and OpenMP specification date (201511), establishing a reproducible baseline for benchmarking.
- Thread-team correctness was confirmed in task1_hello.c: omp_get_num_threads() correctly reported the team size exactly once (via single), and omp_get_thread_num() produced unique, race-free rank identification for every thread (via critical), validating the fork-join model and mutual-exclusion primitives.
- Empirical benchmarking across p=2,4,8,16 threads on a small (N=1000) sum-of-squares workload showed sub-linear speedup peaking at S(8) ≈ 2.89 before regressing at p=16 to S(16) ≈ 2.70 — a direct, measured illustration of Amdahl's Law and of thread-management/barrier overhead overtaking available parallel work as granularity per thread shrinks, compounded by scheduling non-determinism across the chip's heterogeneous P-core/E-core layout.
- Theoretical peak FLOPS analysis established hard upper bounds on this processor's floating-point throughput: 1.1264 TFLOPS (SP) / 0.5632 TFLOPS (DP) at base clock, rising to 2.7648 TFLOPS (SP) / 1.3824 TFLOPS (DP) at max turbo — figures that serve as the theoretical ceiling against which any future compute-bound benchmark's sustained FLOPS should be normalized to compute efficiency relative to peak.

Overall, the experiment demonstrates that effective use of shared-memory parallelism requires co-reasoning about algorithmic parallelism (Amdahl's Law), workload granularity relative to synchronization overhead, and the underlying hardware's SIMD/FMA throughput ceiling — three concerns that together define the practical limits and design space of HPC application performance on modern hybrid multi-core CPUs.

End of Report — Experiment 1